Question 1 [14]

a) What does the following algorithm do? [1]

```
ALGORITHM mystery(int x,int y)
   while y ≠ 0 do
     t ← x mod y
     x ← y
     y ← t
   return x
```

Answer: It computes the gcd of x and y.

b) Explain your answers in each case below:

i) Is $O(n!) \subseteq O(2^n)$? [2]

ii) Is $O(n \log_2 n) \subseteq O(n!)$? [2]

iii) Is $\Theta(n^3) \subseteq \Theta(n^2)$? [2]

Answer

i) No (1 mark). n! grows faster than 2^n (need to make an argument for growth rate). (1 mark)

ii) Yes (1 mark) . $n\log_2 n$ grows slower than n! (provide some kind of argument for this) but big Oh provides an upper bound. (1 mark)

iii) No (1 mark). n^3 grows faster than n^2 (make an argument for this) so an n^3 function is not in $\Theta(n^2)$. So there are elements of $\Theta(n^3)$ that are not in $\Theta(n^2)$. (1 mark)

c) Consider the following algorithm:

```
ALGORITHM whoknows(int x,int y, int z, int a)
  if a > 0 do
    whoknows(x,z,y,a-1)
    print x, z, a
    whoknows(y,x,z,a-1)
```

i) What does the algorithm above do? [1]

ii) Set up a recurrence relation for the number of basic operations being perfomed and solve it. Show all your work. [3]

Answer:

i) It solves the Towers of Hanoi problem. (1 mark)

ii) T(0) = 0. (1 mark) T(n) = T(n-1) + 1 + T(n-1) = 2T(n-1) + 1 = 2(2T(n-2) + 1) + 1 =… 2^kT(n-k) + (2^k-1). (1 mark) Set k=n. Then T(n) = 2^nT(0) + 2^n-1 = 2^n-1 (1 mark)

d) Consider the two algorithms below. Are they in the same efficiency class or not? Justify your answer. [3]

```
ALGORITHM A2(int n > 0)
if n=1 return 1 else return 1 + A2(n/2)

ALGORITHM A5(int n > 0)
if n=1 return 1 else return 1 + A5(n/5)
```

Answer:

Yes they are. (1 mark) A2 performs roughly $\log_2 n$ basic operations and A2 performs roughly $\log_5 n$ basic operations. (1 mark) Logarithmic functions grow at the same rate, regardless of the base (need to provide some argument for this). (1 mark)

Question 2 [10]

a) Strassen's algorithm for matrix multiplication make use of the fact that two 2x2 matrices can be multiplied as below. Use this result to write an algorithm in pseudocode for matrix multiplication of an n x m matrix by an m x k matrix (yielding an n x k matrix). [8]

$$\begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix} \times \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{bmatrix} = \begin{bmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{bmatrix}$$

$$\begin{bmatrix} m_1+m_4-m_5+m_7 & m_3+m_5 \\ m_2+m_4 & m_1+m_3-m_2+m_6 \end{bmatrix}$$

$$\begin{aligned} m_1 &= (a_{00}+a_{11})*(b_{00}+b_{11}) \\ m_2 &= (a_{10}+a_{11})*b_{00} \\ m_3 &= a_{00}*(b_{01}-b_{11}) \\ m_4 &= a_{11}*(b_{10}-b_{00}) \\ m_5 &= (a_{00}+a_{01})*b_{11} \\ m_6 &= (a_{10}-a_{00})*(b_{00}+b_{01}) \\ m_7 &= (a_{01}-a_{11})*(b_{10}+b_{11}) \end{aligned}$$

Answer:

Fill A and B up with zeroes so as to get two n x n matrices where n is a power of 2. (1 mark)
Then apply the algorithm below.

```
STRASSEN(A, B):

If n = 1 Output A00 × B00 (½ mark)

Else

  Split matrices into 8 n/2 X n/2 parts: A00 , B00 , . . . , A11 , B11

      (1/2 mark)

  M3 = Strassen(A00, B01 – B11) ½ mark

  M5 = Strassen(A00 + A01 , B11 ) ½ mark

  M2 = Strassen(A10 + A11 , B00 ) ½ mark

  M4 = Strassen(A11 , B10 – B00 ) ½ mark

  M1 = Strassen(A00 + A11 , B00 + B11 ) ½ mark

  M7 = Strassen(A01 – A11 , B10 + B11 ) ½ mark

  M6 = Strassen(A10 – A00 , B00 + B01 ) ½ mark

  C11 = M1 + M4 – M5 + M7 ½ mark

  C12 = M3 + M5 ½ mark

  C21 = M2 + M4 ½ mark

  C22 = M1 + M3 – M2 + M6 ½ mark

  Output C ½ mark
```

b) Consider the algorithm below. What is the efficiency class of this algorithm in the worst case? Explain what the input will look like in the worst case. [2]

```
ALGORITHM X(A[0..n-1])
  for i=1 to n-1
    v = A[i]
    j = i-1
    while j ≥ 0 and A[j]>v do
      A[j+1] = A[j]
      j = j – 1
```

```
A[j+1] = v
```

Answer: Efficiency class in the worst case is $\Theta(n^2)$. (1 mark) This will occur if the elements in A are in decreasing order. (This is insertion sort.) (1 mark)

Question 3. **[4]**

Consider the following pseudo-code to find the shortest distance between a set of points on a Euclidean plane, each with an x and y coordinate.

```
1.  Sort the points by x order (tiebreak on the y value) into P.
2.  Sort the points by y order (tiebreak on the x value) into Q.
3.  EfficientClosestPair(P, Q)
4.  EfficientClosestPair(P, Q):
5.    if n <= 3: return minimal distance found by brute force
6.    copy the first ceiling[n/2] points of P to Pl
7.    copy the same points of Q to Ql
8.    copy the remaining floor[n/2] points of P to Pr
9.    copy the same points of Q to Qr
10.   dl = EfficientClosestPair(Pl, Ql)
11.   dr = EfficientClosestPair(Pr, Qr)
12.   …..
13.   m = P[ceiling(n/2)]-1].x
14.   copy all points of Q for which |x-m|<d into S[0..num-1]
15.   dminsq = d * d
16.   for i = 0 to num – 2:
17.     k = i + 1
18.     while k <= num – 1 and (S[k].y – S[i].y)^2 < dminsq:
19.       dminsq = min((S[k].x-S[i].x)^2+(S[k].y-S[i].y)^2, dminsq)
20.       k = k + 1
21.   return sqrt(dminsq)
```

a. Complete line 12. [1]

Answer: d = min(dl, dr) (1 mark)

b. Lines 16 to 20 appear to be $\Theta(n^2)$ in the worst case. Explain why this isn't true, and what the worst case efficiency actually is for these lines of code. [3]

Answer: The inner while loop executes at most five times because there can be at most six points in the region of the plane under consideration. (1 mark) Consequently lines 17 to 19 are in a constant efficiency class. (1 mark) The efficiency class for these lines of code is determined by the for loop on line 15 which is $\Theta(n)$. (1 mark)

Question 4. **[8]**

Consider the following code:

```
1.  X(A[l .. r]):
```

```
2.    p = A[l]
3.    i = l
4.    j = r + 1
5.    repeat:
6.      repeat ++i until A[ i ]>= p
7.      repeat --j until A[ j ]<= p
8.      swap(A[ i ], A[ j ])
9.    until i >= j
10.   swap(A[ i ], A[ j ])
11.  swap(A[l],A[j]
12. return j
```

a. Explain what the algorithm does and give a high-level description of how it accomplishes this. [3]

b. The algorithm contains a bug in lines 6 and 7. Explain what the bug is and how to fix it. [2]

c. Explain why line 10 is necessary. [2]

Answer:

a. Rearranges the list so that it is partitioned into those elements smaller (or equal to) the pivot element p (the value initially in A[l] and those greater or equal to p, with those ≤ p to the left of p, and those ≥ p to the right of p (but not necessarily sorted). (1 mark) It accomplishes this by scanning the list simultaneously from the left, looking for the first value ≥ p (using the index i) and from the right, looking for the first value ≤ p (using the index j), and swapping these. (1 mark) This continues until the indices cross (or are equal). (1 mark) Some explanation such as this.

b. Problem in line 6 if all values in A are less than p. (1/2 mark) Similarly, problem in line 7 if all values are greater than p. (1/2 mark). Solution:

repeat ++i until A[i]>= p or i==r (1/2 mark)

repeat --j until A[j]<= p or j==l (1/2 mark)

c.

If i==j the swap makes no difference (it doesn't swap anything). (1/2 mark) If i>j we will have that A[j] ≤ A[i] before the last swap in the outer repeat loop, and therefore A[j] ≥ A[i] after exiting the outer repeat loop. (1/2 mark) This means we may end up with a situation for which A[j] > A[i], and therefore we could have A[j] > p or A[i] < p. (1/2 mark) Swapping A[l] and A[j] would then produce incorrect results. (1/2 mark) By swapping A[i] and A[j] back just outside of the outer repeat loop we ensure that the new A[i] ≥ p and the new A[j] ≤ p, ensuring that the swapping of A[l] and A[j] produces the correct result. (1 mark)

Question 5. **[5]**

Prove, using the formal definitions of Deterministic Finite Automata and Non-deterministic Finite Automata, that for every Deterministic Finite Automaton there is a Non-deterministic Finite Automaton that accepts exactly the same language.

Answer: Let D = (Q, Sigma, delta, q0, F) be a DFA. (1 mark) We need to find an NFA N such that N accepts the same language as D. (1 mark) We define N as: (Q,Sigma,delta', q0, F) (1 mark) where q' is defined as follows:
For every q in Q and every s in Sigma, delta'(q,s) = {delta(q,s)} (1 mark)
For every q in Q, delta'(q,epsilon) = emptyset (1 mark)